A Model For Smarter Urban Intersections

A Science Research Paper Presented to

Newton Country Day School Science and Engineering Fair

February 27, 2024

Ludovica Melodia

Mrs. Haddad

Grade 11

Table of Contents

Acknowledgments…….....2

Abstract.…….3

Introduction…….……4

Experimental Design……...…….....9

Data and Results……..24

Discussions and Conclusions……...……32

Bibliography……..……...…35

Acknowledgements

I would like to thank the Newton Country Day science department for giving me this opportunity to undertake this project. I would especially like to thank Mrs. Haddad, my mentor throughout this process, for helping me overcome all of the obstacles which I was faced with as well as keeping me organized. I would also like to thank my parents for supporting me throughout this process; my father for helping me fix errors in my code and my mother for assisting me with countless things, whether driving me to pick up materials or helping me memorize my presentation. I also want to thank my mother for helping me run my trials. I would not have been able to perform my testing process correctly without her.

Abstract

There are millions of traffic lights throughout the world that are always on, constantly consuming energy. The energy that is used from all of these traffic lights impacts the environment. Energy waste is a prominent issue in today's world, which is why it should be an important societal goal to work towards energy efficiency.

Internet of Things (IoT) is a concept with which objects in our environment can communicate with each other to create more efficient systems. Last year, I created an IoT model for a traffic light which only turned on when it could sense a car in range. Unlike other already existing sensor operated traffic lights, the light in my model communicated with cars using Bluetooth, enabling it to have an accurate sense of where each nearby vehicle was. This year, I expanded on this model to add a pedestrian component. I used two Arduino circuit boards with WiFi capabilities. One board, representing a car or person, has a GPS attachment, and sends its speed to the second board, representing the traffic light. If the speed received is that of a pedestrian, the light cycle starts green for the pedestrian and red for the vehicle, and then changes to allow the vehicle to pass after a set number of seconds. If the speed received is that of a vehicle, the lights cycle through in the opposite order.

I tested the model in 20 trials, 10 for the vehicle component and 10 for the pedestrian component. In these trials, I recorded whether or not the correct lights turned on based on the speed of the GPS. In all of my trials, the model worked without error. My hope for the future is that this model can be implemented in real traffic situations to make urban environments more efficient.

Introduction

About two thirds of the energy that is produced each year in the US is wasted (Webber and Rhodes, 2017). The exaggerated use of energy in today's society is a large contributor to the pollution and overall destruction of the Earth. While the use of environmentally friendly ways to create energy is increasing, the three most common energy sources are natural gas, coal, and nuclear power; each of these harms the planet. Natural gas and coal are nonrenewable resources which our current society consumes at alarmingly rapid rates, while nuclear power comes with many risks, and produces radioactive waste.

The U.S. electricity system has a tremendous environmental impact - it causes greenhouse gas emissions, excessive use of water, water pollution, and generation of solid waste. In addition, the production of electricity takes up vast amounts of land, and harms the ecosystems of this land (EPA, 2022). It is important to develop solutions to make everyday life more energy efficient, in order to avoid all of these effects which are harming the Earth. One area in which energy usage could be reduced concerns traffic lights.

Traffic lights are on all the time, a constant consummation of energy. Even in certain times or areas in which there are usually no cars, traffic lights cannot be turned off in the eventuality that a car does come through. Each year, an estimated 3 billion kilowatts of energy are consumed on traffic lights (Fama Traffic, 2020). Additionally, idling cars in traffic account for an estimated 30 million tons of carbon dioxide annually. During 2020, when the world went into lockdown, daily carbon dioxide emissions decreased by an average of 8.6%. About half of this emissions change was just from surface transportation. Carbon dioxide is a greenhouse gas, and the immense amount of greenhouse emissions caused by vehicles furthers the environmental impact of inefficient road systems. Traffic lights consume unnecessary amounts of energy by

forcing cars to wait for too long, and by remaining on even in the late hours of the night and in deserted streets.

Inefficient traffic lights waste not only energy, but also time. About 20 percent of all driving time is spent waiting at red lights, where the average American will spend a total of 6 months of their life (Taylor, 2020). They can also cause road accidents, which are the leading cause of death in the US. Each year, the over 6 million average passenger car accidents in the country result in more than 38,000 people losing their lives.

Major traffic lights, in cities and areas with dense traffic, operate on a timer. The color of the lights cycles through red, yellow and green lights in timed intervals in order to keep the flow of cars even from all directions. These traffic lights run on pre-programmed systems and do not have sensors, and therefore do not run based on the number and movement of cars that are in the area at any given moment. Traffic lights in more minor areas often have sensors which tell them when to change the signal for an approaching car. Examples of sensors that are used in these types of lights are lasers, video cameras, or “inductive loop systems” - wires in the road that can sense when a car is present. In these smaller areas, the lights on the main road tend to stay on green, and when the sensors sense a car on the side road, the lights change to allow it to pass. These sensors do not always consider cyclists and pedestrians, and the weather can interfere with them (Wonderopolis, n.d.).

Pedestrian traffic lights usually follow the cycles which vehicle ones follow, designed to allow people to cross when there are no cars arriving. Some intersections have actuated signal controls, which are connected to the “walk” button at crosswalks. Pressing this button signals to the system that there is a person waiting to cross, and the lights account for that person, but only after they have finished cycling through the cycle that they are in when the button is pressed. So,

the person pressing the crosswalk button at an intersection is incorporated in the following cycle of lights. Sometimes, these buttons are deactivated if there is a heavy traffic flow in a particular place or time of day. Like with vehicle lights, pedestrian signals run on fixed cycles in more urban areas.

Right now, only three percent of all traffic lights in the United States are considered "smart"(Austin 2019). While some minor traffic lights can sense the presence of a car, none are able to communicate with and receive signals from cars. In order to do this, they would require wireless communication technology. Wireless communication systems are used for different objects, such as televisions with remotes and GPS systems, to communicate without a physical medium. Instead of using a cable to carry the signals like in wired communication, wireless communication uses Electromagnetic (EM) Waves - typically Microwave Rays and Radio Waves. These systems function with three main components: a transmitter, a channel, and a receiver. The transmitter is the source of the signal, and where the signal is encoded, encrypted, modulated and multiplexed in order to be transmitted through an antenna. The channel, or medium of transmission of the signal, is where the signal travels. Wired communication functions with a guided medium, such as cables, for the signal to travel on. Wireless communication, however, uses an unguided medium - it is called this way because the signal propagates through space without any guidance. These waves do not have a distinct shape or path, which is why there cannot be a guide made for them. The final component is the receiver. The receiver basically has to take the code from the channel and transform it back into what it was before it went through the transmitter. The signal must go through Demodulation, Channel Decoding, Decryption and Source Decoding. The final step, the source decoder, is what brings the signal back to the original message (Electronics hub, 2021).

One type of wireless communication technology is Wi-Fi. Wi-Fi can use two radio wave frequencies, either 2.4 gigahertz or 5 gigahertz. One gigahertz corresponds to one billion frequency waves per second. The higher the frequency, the greater the amount of data transmitted per second. Devices, like computers, send requests via the internet in the form of binary code (combinations of one and zero). Wi-Fi chips in the devices then send these frequencies through one of the two radio channels mentioned above to a router, which translates them and allows the device to perform the requests, such as loading a file. This process happens incredibly fast, usually at 54 megabits per second.

MQTT (Message Queuing Telemetry Transport) is a protocol with which information can be sent from one device (a sender) to another (receiver) through a WiFi broker. The “publisher” can publish information, such as sensor data, to a specific topic. This information is then sent through the MQTT broker, and to a “client”, which subscribes to the topic to receive the information being sent in it. MQTT has a number of benefits. Its clients are designed to be efficient and can be used on small microcontrollers. It allows for bi-directional communications, and is also security enabled and easy to encrypt.

GPS (Global Positioning System) is its own type of radio, and the system consists of over 30 satellites which circle the Earth. These navigation satellites send constant signals, which can be received by GPS receivers such as the ones in smartphones. Once a GPS receiver obtains a signal from multiple satellites, it can calculate its position based on its distance from the satellites at that specific time. A receiver requires signals from at least four satellites before it can accurately calculate location. GPS location data is in the format of latitude, a measurement north or south based on distance from the Equator, and longitude, a measurement of east or west based

on distance from the Prime Meridian. These measurements create a coordinate system which is used to express position on the whole planet, in decimal degrees.

There are already working solutions to create more efficient traffic systems to reduce energy waste. One of these is a system that is being developed and put into effect in some areas for “smart traffic lights.” In this system, created by a company called Surtrac, Artificial Intelligence (AI) traffic lights can communicate with each other. These lights use sensors and cameras to gather real time information about what is happening on the street and then send this information to one another to create the most efficient traffic patterns. With this system, the traffic signals in any given place can consider specific scenarios. For example, if a person in a wheelchair is crossing the street, a camera could sense this and let the nearby lights at its intersection know that they will need to stay on for some time longer. The company found that with this network of smart traffic lights, travel time decreased by 25-26% (Austin, 2019). Another model was developed for a smart traffic light system in which traffic lights can sense cyclists through signals from their cell phones (Anagnostopoulos et al., 2016). There is also a campaign to replace the lights in traffic lights with LEDs, which consume less energy (Fama Traffic, 2020). LED bulbs use between 6 and 22 watts, while incandescent bulbs require 135 watts. In a study that tested traffic lights with LED bulbs versus incandescent bulbs in two similar intersections over a period of four months, it was found that the LEDs consumed an average of 111 kWh per month, while the incandescent lights consumed an average of 1203 kWh per month. Using LEDs also has financial benefits - while the bulbs cost more, the significantly smaller energy consumption results in a much smaller energy bill (anonymous, n.d.).

In sum, the current system for traffic lights is outdated and wasteful.

Experimental Design

Materials:

- 2 Arduino Uno WiFi Rev2 circuit boards
- Arduino NEO-6M GPS module
- 2 green, 1 yellow, 4 red, 2 black jumper wires
- 5 330Ω resistors
- 2 red, 1 yellow, 2 green 10 mm LED lights
- 2 USB 2.0 type B to USB-C cable connectors

Tools:

- 2 Computers with Arduino IDE software, version 2.2.1
- Car (for testing process)

Building Process

- Take the 5 colored jumper wires and the Arduino board
- Connect one red wire from pin 2 on the Arduino to 5i on the breadboard
- Connect the second red wire from pin 4 on the Arduino to 9i on the breadboard
- Connect the yellow wire from pin 6 on the Arduino to 14i on the breadboard
- Connect one green wire from pin 8 on the Arduino to 5b on the breadboard
- Connect the second green wire from pin 10 on the Arduino to 14b on the breadboard
- Take the two black jumper wires
- Connect the black wire from the GND pin on the Arduino to the topmost negative pin on the breadboard

- Connect the second black wire from one negative pin on the breadboard to one on the the other side
- Take the remaining two red jumper wires
- Connect one red wire to the topmost positive pin on the breadboard and to the 5V pin on the Arduino
- Connect the final red wire between one positive pin on the Arduino and one on the other side
- Take the 5 resistors and put each with one side in the breadboard pin directly below a colored wire, and the other side in a negative pin
- Take the 5 LED lights
- Put one red light with the longer side in pin 5h and the shorter side in pin 6h
- Put the other red light with the longer side in pin 5c and the shorter side in pin 6c
- Put the yellow light with the longer side in pin 9h and the shorter side in pin 10h
- Put one green light with the longer side in pin 14h and the shorter side in pin 15h
- Put the other green light with the longer side in pin 14b and the shorter side in pin 15b
- Connect the battery to the clip connector and plug it into the Arduino board
- Connect the Arduino board to a computer with a USB cable
- Connect the NEO-6M GPS module to the second Arduino
- Upload the following subscriber code onto the Arduino board with the lights, and the client code onto the Arduino board with the GPS, using the Arduino IDE software

Subscriber Code:

```
#include <ArduinoMqttClient.h>
#include <WiFiNINA.h>
#include "arduino_secrets.h"
#include <arduino.h>

///////please enter your sensitive data in the Secret
tab/arduino_secrets.h
char ssid[] = SECRET_SSID;        // your network SSID
char pass[] = SECRET_PASS;    // your network password

// Traffic light pins
const int redCARPin = 2;
const int yellowCARPin = 4;
const int greenCARPin = 6;
const int redPEDPin = 8;
const int greenPEDPin = 10;

WiFiClient wifiClient;
MqttClient mqttClient(wifiClient);

const char broker[] = "test.mosquitto.org";
int        port     = 1883;
const char topic[]  = "latitude";
const char topic2[]  = "longtitude";
const char topic3[] = "speed";
float speeed;

bool traffic = false;

void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(9600);

  pinMode(redCARPin, OUTPUT);
  pinMode(yellowCARPin, OUTPUT);
  pinMode(greenCARPin, OUTPUT);
  pinMode(redPEDPin, OUTPUT);
```

```
  pinMode(greenPEDPin, OUTPUT);

  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB
port only
  }
  // attempt to connect to Wifi network:
  Serial.print("Attempting to connect to SSID: ");
  Serial.println(ssid);
  while (WiFi.begin(ssid, pass) != WL_CONNECTED) {
    // failed, retry
    Serial.print(".");
    delay(5000);
  }

  Serial.println("You're connected to the network");
  Serial.println();

  Serial.print("Attempting to connect to the MQTT broker: ");
  Serial.println(broker);

  if (!mqttClient.connect(broker, port)) {
    Serial.print("MQTT connection failed! Error code = ");
    Serial.println(mqttClient.connectError());

    while (1);
  }

  Serial.println("You're connected to the MQTT broker!");
  Serial.println();

  // set the message receive callback
  mqttClient.onMessage(onMqttMessage);

  Serial.print("Subscribing to topic: ");
  Serial.println(topic);
  Serial.println();

  // subscribe to a topic
```

```
  mqttClient.subscribe(topic);
  mqttClient.subscribe(topic2);
  mqttClient.subscribe(topic3);

  // topics can be unsubscribed using:
  // mqttClient.unsubscribe(topic);

  Serial.print("latitude: ");
  Serial.println(topic);
  Serial.print("longtitude: ");
  Serial.println(topic2);
  Serial.print("speed: ");
  Serial.println(topic3);

  Serial.println();
}

void loop() {
  // call poll() regularly to allow the library to receive MQTT
messages and
  // send MQTT keep alive which avoids being disconnected by the
broker
  mqttClient.poll();
}

  void onMqttMessage(int messageSize) {
    // we received a message, print out the topic and contents
    Serial.println("Received a message with topic '");
    Serial.print(mqttClient.messageTopic());
    Serial.print("', length ");
    Serial.print(messageSize);
    Serial.println(" bytes:");

if (traffic == false) {
  if (mqttClient.messageTopic() == "speed") {
    Serial.print ("received a speed message");
    // use the Stream interface to print the contents
    int i = 0;
    char velo[30];
```

```
    while (mqttClient.available()) {
    // Serial.print((char)mqttClient.read());
      velo[i] = (char)mqttClient.read();
      i++;

    }
    Serial.print ("here is the speed");
      Serial.print (velo);

    Serial.println();
    Serial.println();

    speeed = atof(velo);
    Serial.print ("here is the speed post conversion");
      Serial.print (speeed);

      lights(speeed);
  }
    else {
      Serial.print ("received a different message");
    // Serial.print ((char)topic[3]);
    }
  }
}

void lights (float speeed) {
traffic = true;
Serial.print ("I am into lights and the speed is: ");
Serial.print (speeed);
    if (speeed < 2.2352) {//(5 mph in meters/second)
    Serial.print ("low speed ");
    digitalWrite(greenPEDPin, HIGH);
    digitalWrite (redPEDPin, LOW);
    digitalWrite (redCARPin, HIGH);
    digitalWrite (yellowCARPin, LOW);
    digitalWrite (greenCARPin, LOW);
    delay (6000);
    digitalWrite (greenPEDPin, LOW);
    digitalWrite (redPEDPin, HIGH);
```

```
    digitalWrite (redCARPin, LOW);
    digitalWrite (yellowCARPin, LOW);
    digitalWrite (greenCARPin, HIGH);
    delay (4000);
    digitalWrite (greenPEDPin,  LOW);
    digitalWrite (redPEDPin, HIGH);
    digitalWrite (redCARPin, LOW);
    digitalWrite (yellowCARPin, HIGH);
    digitalWrite (greenCARPin, LOW);
    delay (2000);
  }
  else if (speeed > 2.2352) {
    Serial.print ("high speed ");
    digitalWrite (greenPEDPin, LOW);
    digitalWrite (redPEDPin, HIGH);
    digitalWrite (redCARPin, LOW);
    digitalWrite (yellowCARPin, LOW);
    digitalWrite (greenCARPin, HIGH);
    delay (4000);
    digitalWrite (greenPEDPin,  LOW);
    digitalWrite (redPEDPin, HIGH);
    digitalWrite (redCARPin, LOW);
    digitalWrite (yellowCARPin, HIGH);
    digitalWrite (greenCARPin, LOW);
    delay (2000);
    digitalWrite(greenPEDPin, HIGH);
    digitalWrite (redPEDPin, LOW);
    digitalWrite (redCARPin, HIGH);
    digitalWrite (yellowCARPin, LOW);
    digitalWrite (greenCARPin, LOW);
    delay (6000);
  }
  traffic = false;
}
```

Client Code:

```
#include <SoftwareSerial.h>
#include "TinyGPS++.h"
#include <ArduinoMqttClient.h>
#include <WiFiNINA.h>
#include "arduino_secrets.h"


TinyGPSPlus gps;
SoftwareSerial SoftSerial(1, 0); // rx = 0, tx = 1
WiFiClient wifiClient;
MqttClient mqttClient(wifiClient);


const char broker[] = "test.mosquitto.org";
int port = 1883;
const char topic[] = "latitude";       // For latitude
const char topic2[] = "longtitude";    // For longitude
const char topic3[] = "speed";


char ssid[] = SECRET_SSID;     // your network SSID (name)
char pass[] = SECRET_PASS; // your network password (use for WPA,
or use as key for WEP)


// Variables for GPS coordinates
double lat1 = 0, lon1 = 0, lat2 = 0, lon2 = 0;
unsigned long prevMillis = 0;
const unsigned long interval = 2000; // Interval for speed
calculation in milliseconds


void setup()
{
 SoftSerial.begin(9600); // GPS baud rate
 Serial.begin(9600);     // Serial port baud rate
```

```
 // attempt to connect to Wifi network:
 Serial.print("Attempting to connect to WPA SSID: ");
 Serial.println(ssid);
 while (WiFi.begin(ssid, pass) != WL_CONNECTED)
 {
   // failed, retry
   Serial.print(".");
   delay(5000);
 }


 Serial.println("You're connected to the network");
 Serial.println();


 Serial.print("Attempting to connect to the MQTT broker: ");
 Serial.println(broker);


 if (!mqttClient.connect(broker, port))
 {
   Serial.print("MQTT connection failed! Error code = ");
   Serial.println(mqttClient.connectError());


   while (1)
     ;
 }


 Serial.println("You're connected to the MQTT broker!");
 Serial.println();
}


void loop()
{
 // call poll() regularly to allow the library to send MQTT keep
alive which
```

```
// avoids being disconnected by the broker
mqttClient.poll();


while (SoftSerial.available() > 0)
 if (gps.encode(SoftSerial.read()));{
      unsigned long currentMillis = millis();
         if (currentMillis - prevMillis >= interval) {


// Calculate distance in kilometers using Haversine formula
double dLat = radians(lat2 - lat1);
double dLon = radians(lon2 - lon1);
double a = sin(dLat / 2) * sin(dLat / 2) +
           cos(radians(lat1)) * cos(radians(lat2)) *
           sin(dLon / 2) * sin(dLon / 2);
double c = 2 * atan2(sqrt(a), sqrt(1 - a));
double distance = 6371 * c * 1000; // Radius of Earth in km


// Calculate speed in m/s
double speed = (distance) / (interval / 1000);

// Update previous GPS coordinates
lat1 = lat2;
lon1 = lon2;
lat2 = gps.location.lat();
lon2 = gps.location.lng();
prevMillis = currentMillis;


// Send latitude and longitude readings to MQTT topics
mqttClient.beginMessage(topic);
mqttClient.print(gps.location.lat(), 6);
mqttClient.endMessage();


mqttClient.beginMessage(topic2);
mqttClient.print(gps.location.lng(), 6);
```

```
 mqttClient.endMessage();


 mqttClient.beginMessage(topic3);
 mqttClient.print(speed, 6);
 mqttClient.endMessage();


 // Print latitude and longitude to serial for debugging
 Serial.print("Latitude: ");
 Serial.println(gps.location.lat(), 6);

 Serial.print("Longitude: ");
 Serial.println(gps.location.lng(), 6);

 Serial.print("Speed: ");
 Serial.println(speed);

  Serial.print("distance: ");
 Serial.println(distance);

 Serial.print("interval: ");
 Serial.println(interval);

 delay(500); // Adjust delay as necessary based on your
application requirements

        }
      }
    }
```

Modification Process of the Design

Last year, I created a model for a traffic light using Bluetooth Low Energy (BLE), where I used an Android phone to represent a moving car, and an Arduino circuit board to represent the traffic light. If the Arduino sensed the "car" in range, it turned the lights on, and when no cars were sensed in range, it turned them off. This year, my goal was to expand my model by adding a pedestrian component to it. I decided that my model would be able to differentiate between a vehicle and a pedestrian based on the speed at which they were moving.

My first idea was to create this model using a phone to represent a car and an Arduino board to represent a traffic light, communicating BLE, as I did with my model last year. However, in last year's model, the two devices did not send actual real-time information with each other. Instead, the light only sensed that the car was in range by automatically establishing a BLE connection with it. In my first attempt, I began to research a way to use BLE to send GPS coordinates to an Arduino. I found an existing module which could be attached to an Arduino and generate GPS coordinates. However, I decided against using this module, because it would have required the Arduino to represent the car, and I wanted to continue my previous design in which there was a phone representing the car.

I wanted to have the phone send its location coordinates to the Arduino by creating an app using MIT App Inventor. In this app, the phone, representing either a person or a car, would send its location coordinates over Bluetooth. I was able to create the app and have it display the coordinates on the screen of the phone, but was unable to establish BLE communications between the phone and the Arduino. I was not able to understand BLE protocol enough to be able to write a working Arduino code which was able to receive messages. One of my main difficulties was that I wanted the phone to be the BLE peripheral (sender) and the Arduino to be

the central (receiver), so that it could receive the speed data and use it to turn on different lights. In all of the examples that I found, more powerful devices, like phones, were used as centrals, and weaker devices, like Arduinos, were used as peripherals.

After weeks of trial and frustration, I decided to search for a different way in which I could wirelessly send location data to an Arduino. I circled back to my original idea of using a GPS module for Arduino. In my new plan, I abandoned my idea of using a phone to represent the person/car, and replaced it with a second Arduino. I also abandoned my original idea of using BLE for wireless communications, and instead found a way in which two Arduinos could establish communications using a WiFi protocol called MQTT.

In my final design, there are two Arduino boards. One has a GPS module connected to it and represents the car/person, and the other has LED lights connected to it and represents the traffic lights of an intersection. The two boards communicate with each other via MQTT.

Modification Process of the Code

Once I changed my design idea from using BLE to using MQTT, I basically had to start over with my code. I was lucky enough to find an example on the Arduino website of a basic MQTT communication between two Arduinos. At first, I used the example code to establish a basic MQTT communication between the two Arduino boards. The goal of this was just to understand how these communications worked. This example sent 3 random numbers from the sender (client) to the receiver (subscriber), which then printed them into the Serial Monitor (output). I also used an online example for the GPS module. This example simply printed the latitude and longitude coordinates of the board into the Serial Monitor.

After my components worked, I began modifying the MQTT example code. The first modification that I made was to include the GPS into the communications. I changed the MQTT topics, which in the example code contained random numbers, to contain the latitude and longitude coordinates produced by the GPS. I was able to establish a system in which one Arduino sent its coordinates to the other, which then printed them into the monitor.

Once the Arduino "traffic light" could receive GPS coordinates, I had to find a way to manipulate the coordinates in order to tell speed. I found an equation called the haversine formula which assumes (though not totally accurately) that the earth is a sphere, and so calculates speed from latitude and longitude coordinates using the earth's radius. I inserted this formula into the sender code, and then modified the code for it to send the object's speed along with its coordinates.

The final component that I had to add to my code was the light mechanism based on speed. I had some trouble adding this to my code, because it involved taking the message that the computer was receiving and turning it into a variable, on which I then based my conditionals to control the lights. This was difficult because the speed was sent in the form of a string, but I had to convert it into a float, so that I could set it as "less than 5 mph" or "greater than 5mph", to regulate whether to allow a pedestrian or vehicle to pass first at the intersection. The second problem that I had was that once I converted the speed to a variable, I needed to allow it to stop updating the speed every second if the light cycle, which lasted 6 seconds, was running. I did this by adding a boolean with which the code checks every second if it should be running the light sequence, and pauses to run the light sequence (12 seconds) when it receives a speed value.

Testing Process

To test my model, I ran two different experiments, each with ten trials. The first test was for the vehicle light component, and the second test was for the pedestrian light component. The goal of these tests was to measure the accuracy of the speed data that was sent and received.

Vehicle light test:

To test the vehicle portion of the bluetooth connections, I tested the speed reported by the devices against the true speed at which they moved. I stood with my traffic light model at the end of my driveway, stayed next to the light while my mother took the GPS module and went to the edge of the driveway with her car. She drove down the driveway, and when the lights changed I recorded the speed reported by my computer. I repeated this process 10 times for 10 trials.

Pedestrian light test:

I began my testing for the pedestrian component of the system by placing my model in the middle of a field and measuring a 20 meter line. In each trial, I began on the far side of the 20 meter line holding the arduino connected to the gps module, and walked at a constant rate towards the traffic light model. I measured the rate at which I walked by holding a metronome beating and 60 BPM (beats per minute), and taking a step for every beat. My mother stood next to the traffic light model, and at the end of 10 seconds, she recorded the speed indicated by the serial monitor of the traffic light. I repeated this process 10 times for 10 trials.

Results and Data

Pedestrian light:

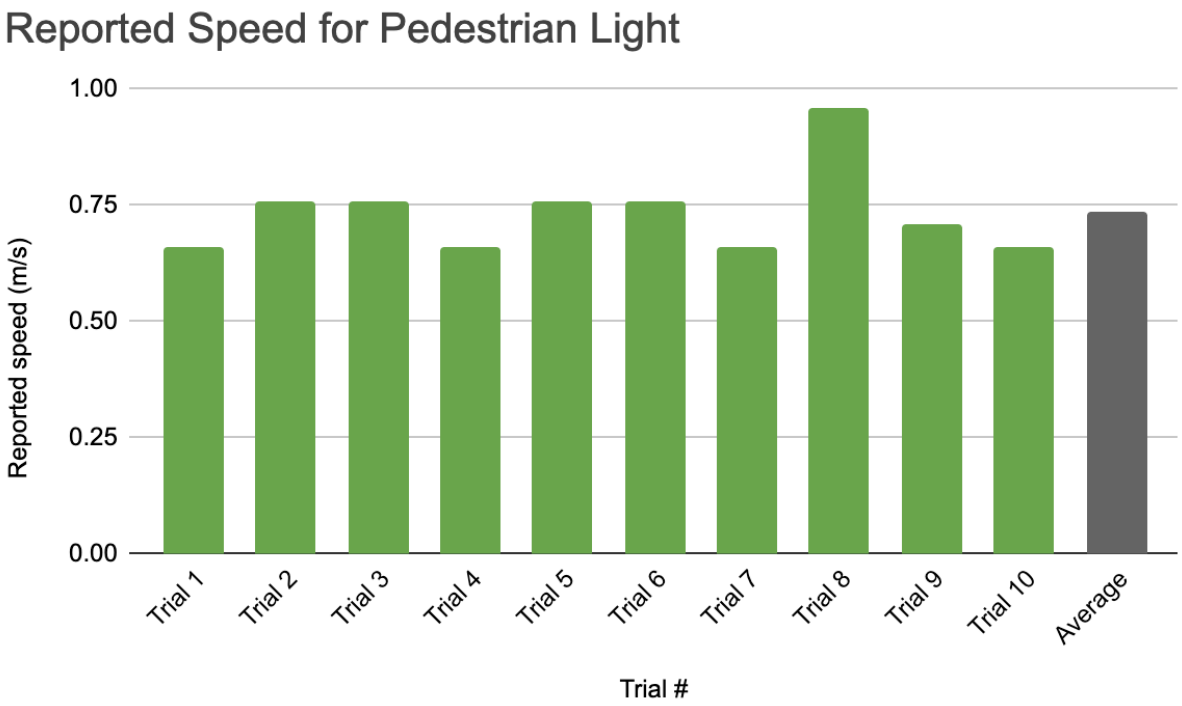
The average speed reported on the Serial Monitor for the pedestrian trials was 0.735 meters per second, with a standard deviation of 0.1 m/s . The average difference between the reported speed and the true speed (.427 m/s) in the pedestrian trials was -0.308 meters per second.

Vehicle light:

The average speed reported on the Serial Monitor for the vehicle trials was 4.288 meters per second, with a standard deviation of 0.5 m/s. The average difference between the reported speed and the true speed (10 miles per hour, or 4.4704 meters per second) in the vehicle trials was 0.1824 meters per second.

Speeds for the Pedestrian Light Trial

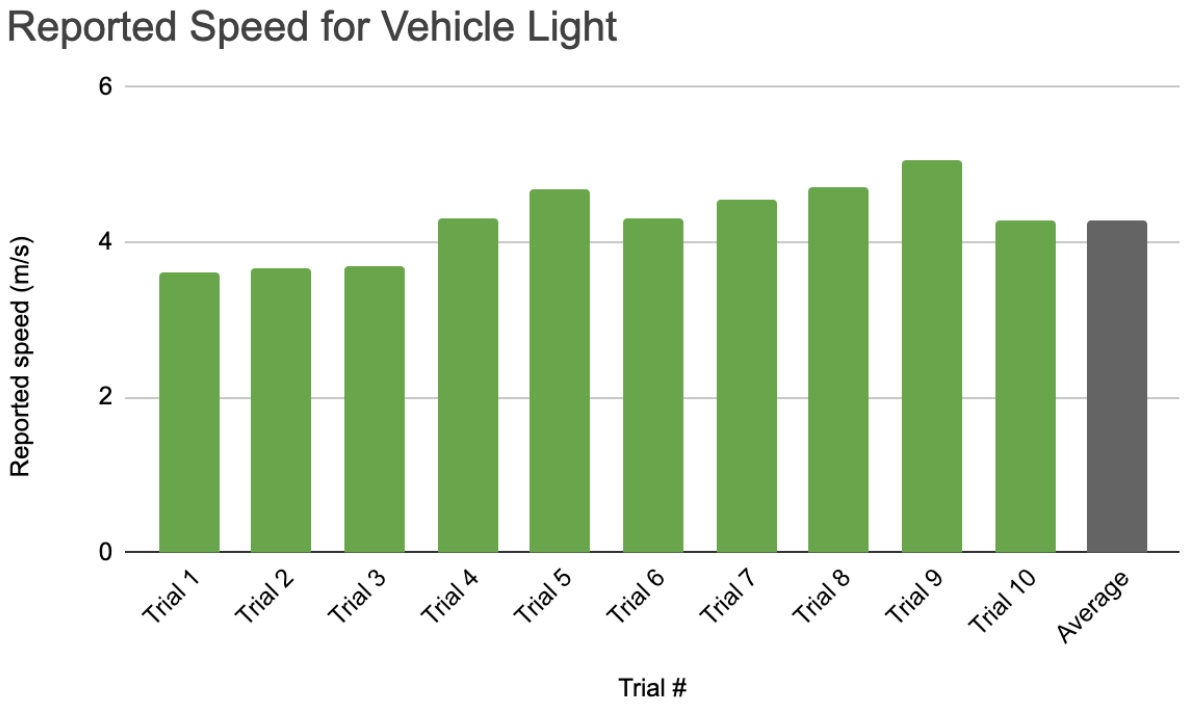
Pedestrian Light	
Trial #	**Reported speed (m/s)**
Trial 1	0.66
Trial 2	0.76
Trial 3	0.76
Trial 4	0.66
Trial 5	0.76
Trial 6	0.76
Trial 7	0.66
Trial 8	0.96
Trial 9	0.71
Trial 10	0.66
Average	*0.735*
Standard Deviation	*0.1*



Speeds for the Vehicle Light Trials

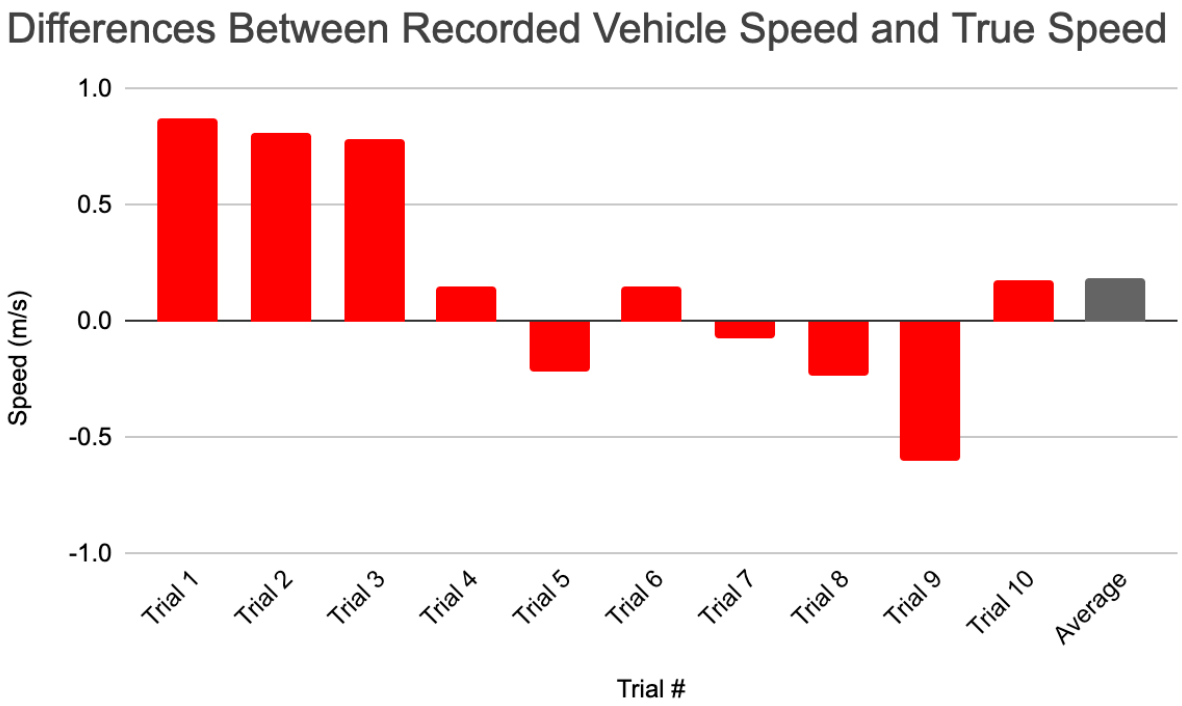
Vehicle Light	
Trial #	**Reported speed (m/s**
Trial 1	3.6
Trial 2	3.66
Trial 3	3.69
Trial 4	4.32
Trial 5	4.69
Trial 6	4.32

Trial 7	4.54
Trial 8	4.7
Trial 9	5.07
Trial 10	4.29
Average	*4.288*
Standard Deviation	*0.5*



Differences Between Reported Speed and Actual Speed in the Vehicle Trials

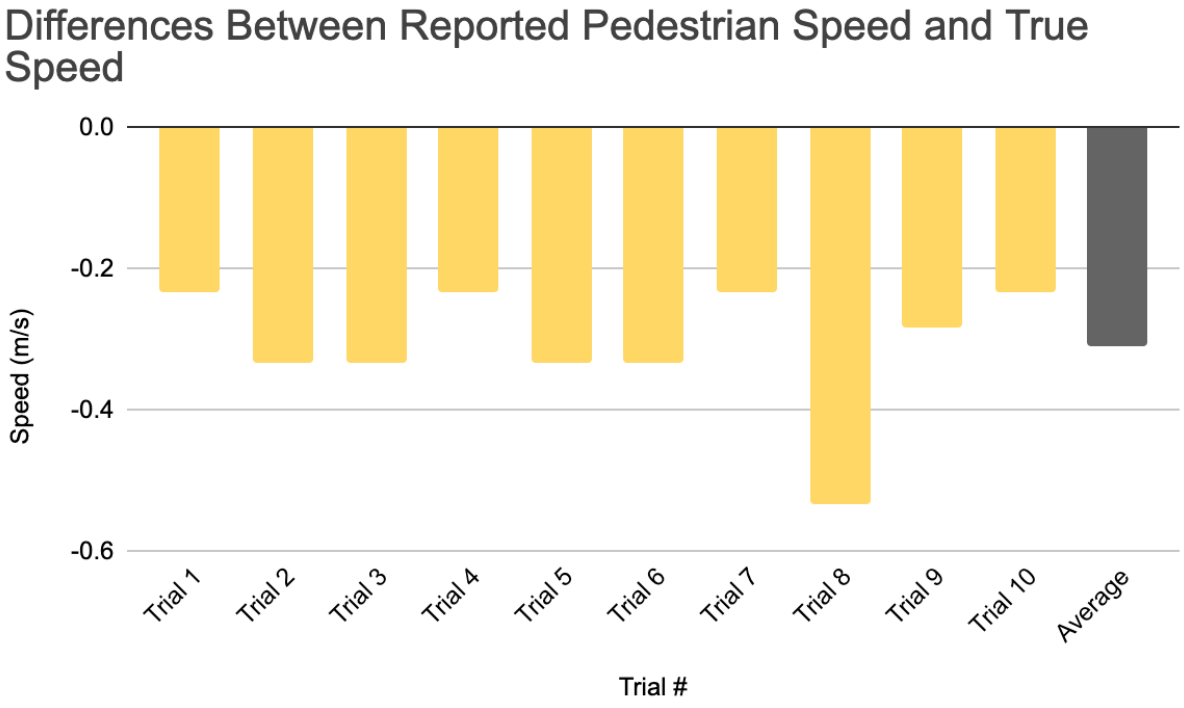
Vehicle Differences	
Trial #	**Speed m/s**
Trial 1	0.8704
Trial 2	0.8104
Trial 3	0.7804
Trial 4	0.1504
Trial 5	-0.2196
Trial 6	0.1504
Trial 7	-0.0696
Trial 8	-0.2296
Trial 9	-0.5996
Trial 10	0.1804
Average	*0.1824*
Standard Deviation	*0.5*



Differences Between Reported Speed and Actual Speed in the Pedestrian Trials

Pedestrian Differences	
Trial #	**Speed m/s**
Trial 1	-0.233
Trial 2	-0.333
Trial 3	-0.333
Trial 4	-0.233
Trial 5	-0.333
Trial 6	-0.333
Trial 7	-0.233

Trial 8	-0.533
Trial 9	-0.283
Trial 10	-0.233
Average	-0.308
Standard Deviation	0.09204467514



Calculations

Equation for average speed: $(S_1 + S_2 + S_3 + S_4 + S_5 + S_6 + S_7 + S_8 + S_9 + S_{10}) / 10$

Standard deviation formula (calculated using the Google Sheets STDEV function):

$$\sigma = \sqrt{\frac{\sum(x_i - \mu)^2}{N}}$$

Walking Speed

- Measure the amount of seconds it takes to walk across a 10 foot line, using a metronome beating at 60 BPM (once every second) → 7.13 seconds
- Convert 10 feet to meters → divide feet by 3.281 → 10 feet = 3.048 meters
- Divide meters/seconds to get speed in meters per second → 3.048 meters/7.13 seconds = .427 meters/second

Driving Speed

- Convert 10 mph to m/s → multiply by 0.447. → 10 mph = 4.4704 m/s

Haversine Formula (derive speed from latitude and longitude coordinates)

Haversine formula:

$$a = \sin^2(\Delta\varphi/2) + \cos(\varphi_1).\cos(\varphi_2).\sin^2(\Delta\lambda/2)$$
$$c = 2.\text{atan2}(\sqrt{a}, \sqrt{(1-a)})$$
$$d = R.c$$

where φ *is latitude,* λ *is longitude, R is earth's radius (mean radius = 6,371km)*

note that angles need to be in radians to pass to trig functions!

JavaScript:

```
var R = 6371; // km
var dLat = (lat2-lat1).toRad();
var dLon = (lon2-lon1).toRad();
var lat1 = lat1.toRad();
var lat2 = lat2.toRad();

var a = Math.sin(dLat/2) * Math.sin(dLat/2) +
        Math.sin(dLon/2) * Math.sin(dLon/2) * Math.cos(lat1) * Math.cos(lat2);
var c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
var d = R * c;
```

Discussion and Conclusions

The goal of this experiment was to create a model of an urban intersection between a vehicle traffic light and a pedestrian crosswalk, with the light cycle changing to allow whichever object it senses first to pass based on its speed.

The reported speed in the trials had an average speed of .735 meters per second for the walking speed, with a standard deviation of 0.1 m/s. The average for the reported vehicle speed was 4.288 meters per second, with a standard deviation of 0.5 m/s . The true walking speed in the trials was .427 meters per second, and the true vehicle speed was 4.4704 meters per second. The average distance between reported speed and true speed was 0.1824 meters per second for the car, and -0.308 for the walking. The true car speed is within one standard deviation of the average reported speed from the trials, which is an indication that the speed recordings were somewhat accurate. However, for the model to be reliable and safe enough to be used in a real traffic situation, the ideal average speed would be the same as the true speed, and the differences would be zero. The pedestrian portion of the model showed more error, with a relatively higher average difference between the recorded speed and the true speed. The true walking speed is more than two standard deviations away from the average reported walking speed. A real traffic system would have to sense the speed accurately, because it would become dangerous if it made mistakes between recognizing a person and a vehicle. However, my design is only a model, and a real traffic light would be much more powerful than an Arduino board. This design could be implemented safely in the future by using different devices and an alternative wireless communication method. The GPS that I used also could have contributed to the recorded inaccuracies, as it was an inexpensive Arduino attachment. Because only ten trials were

performed for each component, this data cannot be considered an exhaustive representation of the model.

The design goal was met by the final model, as it was able in all of my trials to record location coordinates and convert these to speed on one circuit board, and then send the speed to a second circuit board which used it to control the traffic light. The lights were also able to implement the 12 second rotation that was written in the code.

Because the code has many components, there are moments in which it does not work accurately, or when the connection between the two devices falters. The Arduinos have to use a WiFi network as their broker to communicate, so the quality of the local WiFi connection can affect the model. In addition, turning on the lights blocks the constant arrival of new speed messages for 12 seconds to allow the lights to cycle through, which has occasionally caused the signals to freeze. There is also a bug for which the first speed message that arrives upon connection contains an extremely high value in the millions, but after that first second of connection the reported speeds become relatively correct.

There are various conditions which could have affected the results of the speed measurements in my trials. The most important possible source of error was that since my models had to remain connected to WiFi, I had to perform my trials very close to my house. This means that the car could not drive for a long distance, and the connection could have been faulty in the beginning of the trials, since the car began all the way at the end of the driveway. I tried to connect the Arduinos to my cell phone's hotspot in order to have more mobility, but I have not yet figured out how to make that connection work. I measured the recorded speeds in the trials against the "true speeds", 10 mph (4.4704 m/s) for the car and .427 meters/second for my walking speed. The car might not have been going at an exact and constant 10 mph for all of the

trials, especially since it was only able to travel over a small distance. My walking speed in the trials might also not have been exactly the same as when I first measured it. Finally, the haversine formula is not a perfect way of calculating speed from location coordinates, because it assumes incorrectly that the earth is a sphere.

In order to improve the performance of my current project, a number of different materials could be used to obtain more accurate MQTT connections. A different wireless communication system that does not require a WiFi broker can be used, so as to be implemented on any street. The model could also be made to work from smartphones or vehicles themselves, because pedestrians and drivers could not be expected to carry around Arduinos in a real world scenario. Ideally, the wireless communications would also have a longer range as to sense connections from farther away and be safe to use with fast-moving vehicles. The wireless communications could be updated and made more secure, in order to ensure that interference from other devices does not affect the performance of the traffic lights. The model could also be expanded to represent a larger intersection with more components, such as two traffic lanes or cyclist crossings. If these changes were implemented, this model could be reproduced on a larger scale so as to actually be applicable to real traffic lights.

Works Cited

Anagnostopoulos, Theodoros, et al. "Cyclist-aware Traffic Lights through Distributed Smartphone Sensing." *Pervasive and Mobile Computing*, vol. 31, Sept. 2016, pp. 22-36, https://doi.org/10.1016/j.pmcj.2016.01.012. Accessed 1 Nov. 2023.

"Arduino GPS Tutorial." *educ8s.tv*, 2015, educ8s.tv/arduino-gps-tutorial/. Accessed 27 Feb. 2024.

"Arduino_MKRGPS." *Arduino.cc*, Arduino, www.arduino.cc/reference/en/libraries/arduino_mkrgps/. Accessed 27 Feb. 2024.

"ARDUINO UNO WiFi REV2." *Arduino CC*, Arduino, store-usa.arduino.cc/products/arduino-uno-wifi-rev2. Accessed 1 Dec. 2023.

"Car Accident Statistics you Need to Know in 2024." *Darrigo and Diaz Attorneys at Law*, www.ddlawtampa.com/resources/car-accident-statistics-you-need-to-know-in-2021/#:~:text=On%20average%2C%20there%20are%20over,losing%20their%20lives%20each%20year. Accessed 27 Feb. 2024.

Greengard, Samuel. "Internet of Things." *Encyclopedia Britannica*, Encyclopædia Britannica, Inc., 20 Nov. 2023, www.britannica.com/science/Internet-of-Things. Accessed 1 Dec. 2023.

Hart, Mikal. "TinyGPSPLus." *GitHub*, GitHub Inc., github.com/mikalhart/TinyGPSPlus/blob/master/src/TinyGPS%2B%2B.cpp. Accessed 27 Feb. 2024.

"How Does a Traffic Light Work?" *Wonderopolis*, National Center for Families Learning, www.wonderopolis.org/wonder/how-does-a-traffic-light-work. Accessed 1 Dec. 2023.

"How Does GPS Work?" *NASA Science Space Place*, NASA, 27 June 2019, spaceplace.nasa.gov/gps/en/. Accessed 27 Feb. 2024.

Lambert, Graham. "Wireless Communication between Two Arduinos." *Circuit Basics*, www.circuitbasics.com/wireless-communication-between-two-arduinos/. Accessed 1 Nov. 2023.

"latitude and longtitude." *Encyclopaedia Britannica*, Britannica, 6 Dec. 2023, www.britannica.com/science/latitude. Accessed 27 Feb. 2024.

"LED traffic signals save money, time and energy." *Fama Traffic*, Yangzhou FAMA Intelligent Equipment Co.,Ltd., 8 May 2020, www.ledtrafficlight.cn/news_view-46.html. Accessed 1 Dec. 2023.

Mapari, Bhushan. "Interfacing GPS Module With Arduino Uno." *Autodesk Instructables*, edited by Autodesk, Inc., www.instructables.com/Interfacing-GPS-Module-With-Arduino-Uno/. Accessed 27 Feb. 2024.

"MQTT: The Standard for IoT Messaging." *MQTT*, MQTT.org, 2022, mqtt.org/. Accessed 27 Feb. 2024.

Mudunkotuwa, Rashmin. "Real Time Data Transfer for IoT with MQTT , Android and NodeMCU." *Medium*, 16 Nov. 2018, rashm1n.medium.com/real-time-data-transfer-for-iot-with-mqtt-android-and-nodemcu-ae4b01f87be4. Accessed 27 Feb. 2024.

Pedestrian Crossing Light Systems. *ELTECH*, Electrotechnics Corporation, elteccorp.com/products/warning-systems/pedestrian-crossing-light-systems/. Accessed 1 Nov. 2023.

"Rectangular Rapid Flashing Beacon (RRFB)." *ELTECH*, Electrotechnics Corporation, elteccorp.com/products/pedestrian-crossing-systems/rectangular-rapid-flashing-beacon-(rrfb)/. Accessed 1 Nov. 2023.

Söderby, Karl. "Sending Data over MQTT." *Arduino Documentation*, Arduino, 2 Feb. 2024, docs.arduino.cc/tutorials/uno-wifi-rev2/uno-wifi-r2-mqtt-device-to-device/. Accessed 27 Feb. 2024.

"SoftwareSerial Library." *Arduino.cc*, Arduino, 2 Feb. 2024, docs.arduino.cc/learn/built-in-libraries/software-serial/#begin. Accessed 27 Feb. 2024.

Taylor, Daniel. "Morning Start: Do you know how much of your life is spent waiting at red lights?" *Keremos Review*, Keremeos Review and Black Press Media, 2 Apr. 2020, www.keremeosreview.com/news/morning-start-do-you-know-how-much-of-your-life-is-spent-waiting-at-red-lights-3917360. Accessed 1 Dec. 2023.